Do We Need the Shadow?

Freezing-Aware Weight Versioning for Pipeline-Parallel BERT Training

Animesh Jain, Rupankar Podder

Research proposal · Systems Engg. for Deep Learning (CS6886)

October 1, 2026

DDG → PipeDream → PipeDream-2BW + FreezeOut → PolicyFRZ

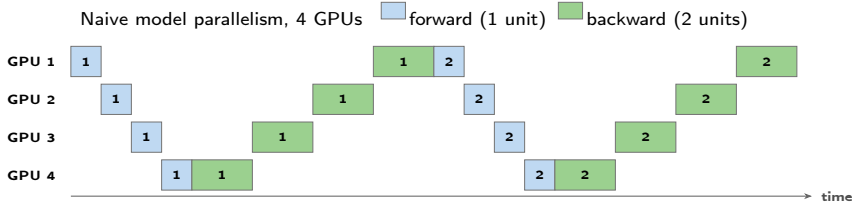
Why training across GPUs stalls

Backprop is a chain of locks

- **Backward locking:** layer $l-1$ waits for layer l: $\delta_{l-1} = \left(\frac{\partial h_l}{\partial h_{l-1}}\right)^\top \delta_l$
- **Update locking:** minibatch $t+1$ needs w^{t+1}
- Model split over K GPUs $\Rightarrow$ **one GPU busy at a time**

Backward is the bigger half

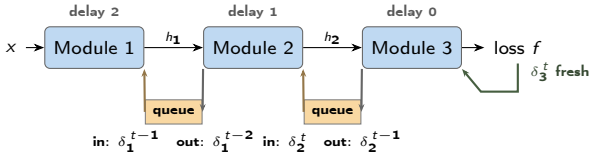
- ResNet-101: forward 53 ms, backward 103 ms
- ResNet-110: forward $\approx$ 32% of a step



Two fixes: **stale gradients** (DDG) and **pipelining** (PipeDream, 2BW). Same side effect: **weights become out of date**.

DDG: decoupled backprop with delayed gradients

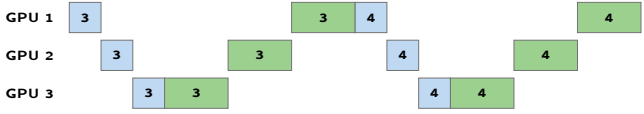
Huo et al., ICML'18



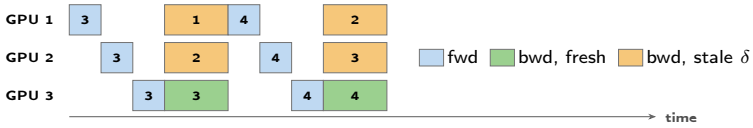
- Split network into K modules on K GPUs
- Forward pass: **unchanged**, sequential
- Module k back-props with the error gradient from iteration $t-K+k$: **delay** $= K - k$
- All modules' backward passes **run in parallel**

$$T_{\text{BP}} = T_F + T_B \quad \longrightarrow \quad T_{\text{DDG}} = T_F + \frac{T_B}{K}$$

Model-parallel BP, K=3 (iterations 3, 4)



DDG, K=3 numbers = sample being back-propagated



DDG: update rule, guarantee, results

SGD-DDG (module k, weights $w_{G(k)}$):

$$w_{G(k)}^{t+1} = w_{G(k)}^{t} - \gamma_t \nabla_{w_{G(k)}} f_{x_{i(t-K+k)}}\big(w^{\,t-K+k}\big)$$

Gradient of an *old* sample at *old* weights, applied to *current* weights. (Adam-DDG is analogous.)

Convergence (non-convex, L-smooth, bounded variance M):

$$\tfrac{1}{T}\textstyle\sum_t \mathbb{E}\|\nabla f(w^t)\|^2 \le \frac{2(f(w^0)-f^*)}{\gamma T} + 2\gamma L M_K$$

$$M_K = KM + K^4 M$$

Error term grows fast with the number of modules K.

2 modules (split at centre), top-1 %

Model	Data	BP	DDG	Δ	modern BP†
ResNet-56	C10	93.12	93.11	−0.01	94.37
ResNet-56	C100	69.79	70.17	+0.38	72.63
ResNet-110	C10	93.53	93.41	−0.12	we run
ResNet-110	C100	71.90	71.39	−0.51	we run

BP baseline is 1.3–2.8 pts below a modern recipe (DDG used lr 0.01 vs. the usual 0.1). We re-run both (Track A).

ResNet-110 / CIFAR-10 across GPUs

	BP	DDG, #GPUs			
	1	1	2	3	4
Top-1	93.53	93.41	93.38	93.39	93.38

≈30–50% less compute time (2→4 GPUs), ∼**2×** **speedup** at 4 GPUs, no accuracy loss *vs. that baseline*.

† **Modern BP: public ResNet-56 CIFAR recipe**, github.com/chenyaofo/pytorch-cifar-models

DDG: what is missing

- **Forward pass still locked.** One minibatch in the forward pipe; best case $T_F + T_B/K$.
  PipeDream: "Huo et al. explored parallelizing the backward pass ... our solution parallelizes both the forward and backward pass."
- **Weight versions are implicit.** The update uses $\nabla f(w^{t-K+k})$, i.e. the weights the *forward* used.
  - Either keep old weights around (memory), or back-prop through *newer* weights (mismatch)
  - The paper discusses neither
- **Staleness grows with K:** delay $K-k$; the bound grows as K^4.
- **Narrow evaluation:** ResNet-8/56/110 on CIFAR, ≤ 4 GPUs in one box, hand-placed modules.

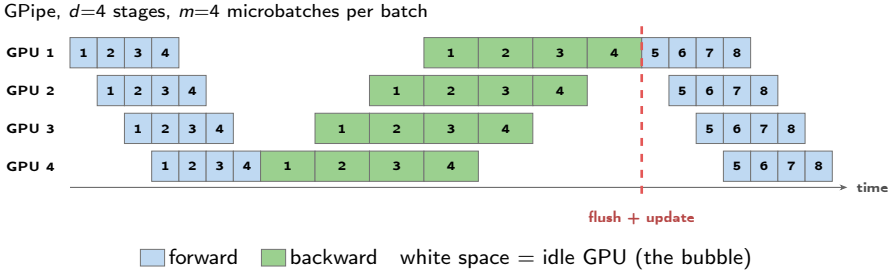
Note: no paper we read shows DDG on BERT or any Transformer. "DDG fails on BERT" is an *open question*, not a result.

$\Rightarrow$ Next: **pipeline** both passes, and make weight versions **explicit**.

Pipelining: keep many (micro)batches in flight

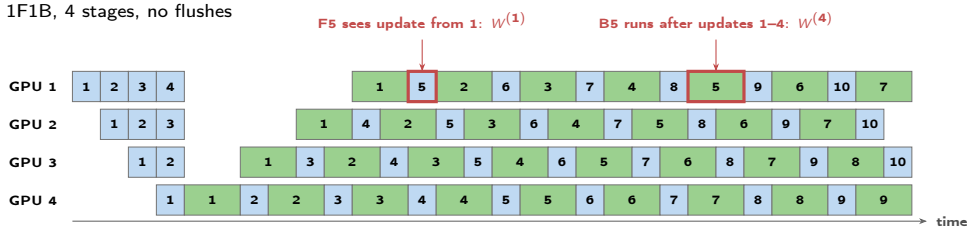
- Cut model into d **stages** (one GPU each); only activations and gradients cross stages
- Split a batch into m **microbatches** and stream them through

- **GPipe**: all forwards, all backwards, accumulate, **flush**, update: $W^{(t+1)} = W^{(t)} - \nu \nabla f(W^{(t)})$, i.e. exact SGD, **1 weight version**
- Cost: idle *bubble* $\approx \frac{d-1}{m+d-1}$ of the time, and activations of all m microbatches stored



PipeDream: 1F1B schedule and the weight-version problem

Narayanan et al., SOSP'19



- Startup: admit d minibatches (NOAM $= \lceil \#\text{GPUs} / \#\text{input-stage replicas} \rceil$)
- Steady state: every GPU alternates **one forward, one backward**: no flushes, no idle
- Each backward **updates its stage's weights immediately**

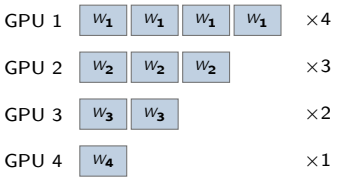
- **Problem:** on GPU 1, minibatch 5's forward and backward use *different* weights
- "not a valid gradient of the loss function f for any vector $w_1, \ldots, w_n$" and it can prevent convergence

PipeDream: weight stashing

- Forward uses the *latest* weights and **stashes** that version; backward of the same minibatch uses the stash
- Result for n stages:

$$w^{(t+1)} = w^{(t)} - \nu \nabla f\left(w_1^{(t-n+1)}, w_2^{(t-n+2)}, \ldots, w_n^{(t)}\right)$$

- Stashing makes each gradient **valid**, **not fresh**: delay depends on the stage, as in DDG
- *Vertical sync* (optional): same version on all stages
- Also: profiler + dynamic-programming partitioner, stage replication (1F1B-RR)
- Memory: $\frac{1}{d}$ of the model $\times$ up to d versions $\approx$ data-parallel footprint



weight versions held at once
(d=4: up to $d-s+1$ on stage s)

PipeDream: results (time to target accuracy vs. data parallel)

Model	Dataset (target)	GPUs	Config	TTA speed-up
VGG-16	ImageNet (68% top-1)	16 (4×4, PCIe)	15-1	**5.28×**
VGG-16	ImageNet	16 (2×8, NVLink)	15-1	2.46×
ResNet-50	ImageNet (75.9% top-1)	16	= DP	1×
GNMT-16	WMT16 En-De (21.8 BLEU)	16 (2×8)	straight	3.14×
GNMT-8	WMT16 En-De	12 (3×4)	straight	2.95×
AWD LM	PTB (98 ppl)	4	straight	**4.25×**
S2VT	MSVD (0.294 METEOR)	4	2-1-1	3.01×

- Accuracy: same target reached in about the **same number of epochs** as data parallel
- 2.6–15× faster than model parallelism; GPipe reaches 35–71% lower throughput
- **Not compared:** DDG. **Not evaluated:** any Transformer

PipeDream: accuracy over time and per epoch

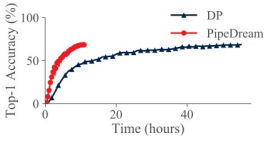


Figure 10: Accuracy vs. time for VGG-16 using 16 GPUs. Each circle or triangle represents two epochs of training.

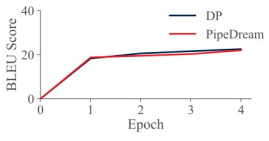


Figure 11: Accuracy vs. epoch using 16 GPUs on Cluster-B.

Left, vs. time (VGG-16, 16 GPUs): PipeDream reaches 68% top-1 much sooner (**5.28**× on cluster A, **2.46**× on cluster B)

Right, vs. epoch: GNMT-16 tracks DP; VGG-16 **lags early**, then catches up. Staleness costs most **early in training**, the same pattern 2BW shows

PipeDream's limits → PipeDream-2BW

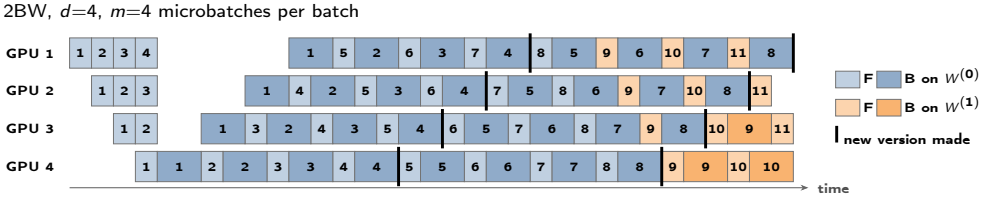
- **Stash memory scales with depth:** up to d weight versions per stage.
  Billion-parameter GPT/BERT on 16 GB V100s: PipeDream **does not fit**, so 2BW could not even run it as a baseline.
- **Stage-dependent delays**; no gradient accumulation inside the pipeline
- **GPipe alternative:** 1 version but flushes, plus activations for all m microbatches
  (GPipe runs out of memory at batch 64 for GPT-2.2B on 8 GPUs)
- **Planner ignores memory** (no activation recomputation); $O(N^3 m^2)$ search

Goal of 2BW

PipeDream's throughput (**no flushes**) with close to GPipe's memory (**few weight versions**).

PipeDream-2BW: double-buffered weights

Narayanan et al., ICML'21



- **Coalesce** gradients over $m \geq d$ microbatches $\Rightarrow$ a new version every m
- In-flight microbatches finish on the **shadow** $W^{(t-1)}$; new ones use $W^{(t)}$
- $\leq$ **2 versions** per stage; old one freed once unused
- Same version for forward and backward, **on every stage**

Microbatch k uses version

$$v(k) = \max\big(\lfloor (k-1)/m \rfloor - 1,\ 0\big)$$

Update rule: **delay exactly 1**, the same on all stages:

$$W^{(t+1)} = W^{(t)} - \nu \nabla f\big(W^{(t-1)}\big)$$

Adam: moments use the 1-stale gradient; no extra optimizer state.

2BW vs. the alternatives, and results

Scheme	versions	flush	update
GPipe / Flush	1	yes	$\nabla f(W^{(t)})$
PipeDream	$\leq d$	no	per-stage delay
2BW	**2**	**no**	$\nabla f(W^{(t-1)})$

Worst-case memory per GPU (with recomputation):

$$\frac{2|W|}{d} + \frac{|A_{\textbf{total}}(b)|}{d} + d\,|A_{\text{input}}(b)|$$

Authors: $|W| \ll |A(b)|$, so "savings from one fewer weight version is *small*."

Convergence (355M, OpenWebText, Adam)

	Vanilla	Van. 90%	2BW
BERT: MNLI	87.77	–	87.82
BERT: RACE	80.06	79.30	79.48
GPT: WT-103 ppl	19.28	–	19.56

Loss matches vanilla **after the first 100k iterations**; overall $\approx$10–20% extra iterations.

Speed (GPT 1.3–3.9B, up to 64 V100s)

- up to **20**$\times$ vs. Megatron tensor MP; **3.2**$\times$ vs. GPipe
- up to 1.6$\times$ vs. PipeDream-Flush; $\sim$30B params on 64 GPUs

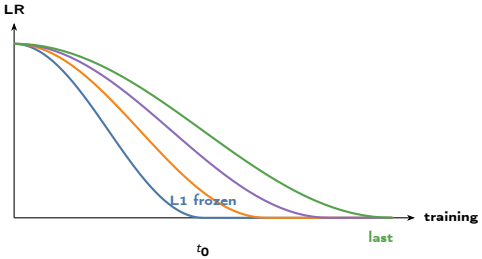
A different lever: layer freezing (FreezeOut)

Brock et al., 2017

- Saves **compute**, rather than parallelising it
- Early layers: most compute, fewest parameters, **converge first**
- Per-layer cosine learning rate, reaching 0 at t_i:

$$\alpha_i(t) = \tfrac{1}{2}\,\alpha_i(0)\big(1 + \cos(\pi t / t_i)\big)$$

  t_i spread from t_0 to the end (linear or cubic)
- At t_i: layer **frozen for good**, dropped from the backward pass
- CIFAR results:
  - DenseNet: up to 20% faster, $\leq 3\%$ relative error increase
  - WRN-40-4: 20% faster, no loss
  - VGG-16: no gain
- Limits: hand-set schedule, one-way freezing, CNNs only



linear schedule, $t_0 = 0.5$

PolicyFRZ: adaptive, reversible freezing



State per layer (window of Δt steps):

$$\mathrm{GV}_i = \frac{\|\sum_j g_i^j\|}{\sum_j \|g_i^j\|} \in [0,1], \quad \mathrm{AM}_i = \tfrac{1}{\Delta t}\sum_j |g_i^j|$$

GV near 1: layer still moving; near 0: gradients cancel (settling).
A PPO policy maps $[\mathrm{GV}, \mathrm{AM}] \in \mathbb{R}^{n\times 2}$ to a type per layer. Reward: $(d_r/n)^2$ if correct, else $-\gamma$.

Results (1× RTX 2080 Ti; training time saved)

- ResNet-50 **28.8%**, VGG-16 22.9%, MobileNetV2 22.0%
- Transformer-base, translation: **27.7%**, BLEU ≈ full training
- Threshold rule alone ($\mathrm{GV}<\theta_1 \wedge \mathrm{AM}<\theta_2$): 27.9%, 92.1 vs. 92.0%

Not tested: BERT, pipelines.

The idea: let freezing decide which layers need a safety copy

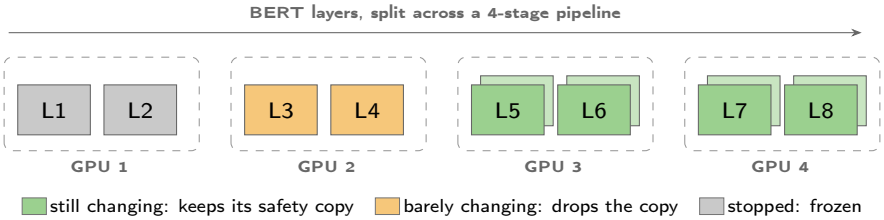
Pipelining (2BW)

Keeps a **safety copy** (the "shadow") of every layer's weights, because weights change while microbatches are still in flight.

Freezing (PolicyFRZ)

Watches each layer's gradients and tells us which layers have **stopped changing**.

Merge: a layer that isn't changing doesn't need a safety copy.



One simple rule per layer, checked every few steps

The layer is...	We...	Gain	Risk
still changing a lot	keep the safety copy (normal 2BW)	none; this is the safe default	none
changing a little	drop the safety copy	one less copy; slightly fresher updates	tiny gradient error
stopped changing	freeze it (no updates)	no copy, **no backward pass**, **no stored activations**	none (can unfreeze)

- "Is it changing?" comes from PolicyFRZ's signal: are the layer's recent gradients **consistent and large**, or **cancelling out and small**?
- Early layers usually settle first, and those sit on the GPU that stores the most in-flight data, so that is where the **biggest memory saving** is

Evaluation plan: three tracks + ablations

A. Re-baseline DDG

ResNet-56/110, CIFAR-10/100

BP vs. DDG with a **modern recipe**

Is staleness really free when the baseline is strong?

B. vs. PolicyFRZ, on its models

ResNet-50, CIFAR-10 (+100), 160 epochs as in PolicyFRZ

Stretch: Transformer-base, Multi30k

Does the merge beat PolicyFRZ where PolicyFRZ was tested?

C. vs. 2BW, on BERT

Size ladder: Mini → Small → Medium → Base (Large: stretch)

Same pipeline depth, same token budget

Does it beat 2BW, and how do gains scale?

Ablations on ResNet-50 (B) and BERT-Medium (C):
switch each component on/off to **attribute every gain**

No model appears in both 2BW and PolicyFRZ, so each parent is compared **on its own models**. All pipeline runs use our exact emulator (2 GPUs); every variant uses the **same recipe and hyperparameters** as its baseline.

Track A: re-checking DDG against a modern baseline

		DDG paper		modern†
Model	Data	BP	DDG	BP
ResNet-56	C10	93.12	93.11	94.37
ResNet-56	C100	69.79	70.17	72.63
ResNet-110	C10	93.53	93.41	*we run*
ResNet-110	C100	71.90	71.39	*we run*

DDG's BP matches the 2016-era ResNet results.

Why the gap matters

- DDG trained *both* arms with lr **0.01**; the standard recipe uses **0.1**
- Staleness error grows with step size (bound $\propto \gamma L M_K$), so parity at lr 0.01 may not hold at 0.1

What we run

- One modern recipe (SGD 0.9, lr 0.1, cosine, standard augmentation) for **both** BP and DDG
- DDG with $K \in \{2, 4\}$ modules
- Report accuracy gap vs. **our** BP, not the paper's

† **Modern BP: public ResNet-56 CIFAR recipe**, github.com/chenyaofo/pytorch-cifar-models

Track B: comparing each parent on its own models

Parent	Its published models	What we run	Compared against
PolicyFRZ	ResNet-50, VGG-16, MobileNetV2 (CIFAR-10/100, ImageNet); Transformer-base (WMT16, Multi30k, IWSLT)	**ResNet-50, CIFAR-10** (+100), 160 epochs; *stretch:* Transformer-base, Multi30k	Published: full 92.0%, threshold rule 92.1% at 27.9% less time (C10); full 76.2% (C100); plus our re-run
2BW	BERT-355M, GPT 355M–3.9B (up to 64 V100s)	**BERT ladder** (Track C)	Our 2BW run on the same model; published 355M results as reference
DDG	ResNet-8/56/110 (CIFAR)	ResNet-56/110	Track A

Why these picks

- No model is shared by 2BW and PolicyFRZ, so we meet each on **its own ground**
- ImageNet (PolicyFRZ, PipeDream) and 355M+ pre-training (2BW) are out of reach on 2 GPUs in 4 weeks

CNN-specific notes

- Early conv layers dominate compute, so freezing saves more than in BERT's uniform layers
- BatchNorm: keep the microbatch size fixed across variants so batch statistics match

Track C: which BERT? A size ladder

Model	Layers	Hidden	Params	Depths d	Protocol
BERT-Mini	4	256	11.3M	2, 4	MLM from scratch + SST-2
BERT-Small	4	512	29.1M	2, 4	MLM from scratch + SST-2
BERT-Medium	8	512	41.7M	2, 4, 8	MLM from scratch + SST-2; **ablation model**
BERT-Base	12	768	110M	2, 4, 6	MLM from scratch (short) + fine-tune SST-2, MNLI
BERT-Large	24	1024	340M	4, 8	*stretch*: fine-tune only; closest to 2BW's 355M BERT

Controlled comparison

- Pre-training on WikiText-103, **same token budget** for every size
- Hold d=4 fixed across sizes, so size is the only variable
- Mini/Small/Medium/Base: Google's compact BERT configs (Turc et al. 2019)

Scalability read-outs (plotted vs. params)

- Loss / accuracy gap to vanilla
- Memory saved per stage (%)
- Fraction of layers frozen or without a shadow over training
- Modelled throughput

Ablations: separating the contribution of each component

#	Configuration	Pipeline	Shadow	Freezing	Isolates
0	Vanilla	none	–	off	reference (no staleness)
1	FRZ only	none	–	GV/AM	freezing alone ($\approx$ PolicyFRZ)
2	2BW	2BW	all	off	cost of the 1-step delay
3	1BW	2BW	none	off	removing every shadow
4	2BW-Sel	2BW	GV/AM	off	selective shadow alone
5	2BW + FRZ	2BW	all	GV/AM	freezing inside a pipeline
6	**Full** (Sel + FRZ)	2BW	GV/AM	GV/AM	**our method**
7	Full, random choice	2BW	random	random	is the GV/AM *signal* needed? (matched counts)
8	Full, fixed schedule	2BW	GV/AM	FreezeOut-style	adaptive vs. scheduled freezing
9	Full, Adam reset	2BW	GV/AM	GV/AM	optimizer state on unfreeze

Attribution (each measured vs. 2BW, row 2):

$$\underbrace{\Delta_6}_{\text{total}} = \underbrace{\Delta_4}_{\text{shadow}} + \underbrace{\Delta_5}_{\text{freezing}} + \text{interaction}$$

Metrics: accuracy / loss, peak memory per stage, modelled time. Run on **ResNet-50/CIFAR-10** and **BERT-Medium**, 2 seeds each.

Questions we want to answer

1. Does DDG still match backprop against a **modern baseline**? Track A
2. Does BERT actually **need the safety copy**? B, C
3. Does the merge beat **PolicyFRZ** on PolicyFRZ's models, and **2BW** on BERT? B, C
4. **Which component** brings which gain, and is the GV/AM signal better than a random choice? ablations
5. How do the gains **scale** from BERT-Mini to BERT-Base (and Large)? Track C

4-week plan and budget

	Tasks	Milestone
Week 1	Pipeline simulator + emulator, DDG emulation, freezing rule; ResNet and BERT-ladder models; time one run of each; start **Track A**	emulator validated; Track A running
Week 2	Finish Track A; **Track B** rows 0–3; **Track C** core runs on Mini, Small, Medium; real 2-GPU check	Track A done; baselines
Week 3	Full method + **ablations** (rows 4–9) on ResNet-50 and BERT-Medium; BERT-Base; Large if time	ablation table
Week 4	Scalability plots, attribution, report, final slides	deliverables

Estimated GPU time (to calibrate in week 1)

- A: 12 ResNet runs $\approx$ 12 GPU-h
- B: 10 configs $\times$ 2 seeds + CIFAR-100 core $\approx$ 36 GPU-h
- C: 4 sizes $\times$ 4 core configs + Medium ablations + fine-tunes $\approx$ 65 GPU-h

$\approx$ **115 GPU-h** $\Rightarrow$ about 2.5 days of both GPUs, inside weeks 2–3 with slack.

If short: 1 seed on the ladder, drop CIFAR-100 and BERT-Large first.

Who does what

	Animesh	**Rupankar**
Week 1	Pipeline simulator + exact emulator (2BW / 1BW weight versions); BERT-ladder models; timing runs	DDG emulation; GV/AM freezing rule + per-layer gradient logging; ResNet models; start **Track A**
Week 2	**Track C** core runs (Mini, Small, Medium); real 2-GPU check of the emulator	Finish **Track A**; **Track B** rows 0–3 on ResNet-50
Week 3	Selective shadow + freezing inside the pipeline; **BERT-Medium ablations**; BERT-Base (Large if time)	GV/AM decision logic, thresholds; **ResNet-50 ablations**; random / fixed-schedule / Adam-reset variants
Week 4	Memory + modelled-throughput scalability plots; systems sections of the report	Attribution analysis; gradient-signal / freezing dynamics plots; method + Track A sections of the report

Shared: common training harness and metrics, weekly result review, final slides.

Summary

- **Pipelining** is fast, but keeps extra weight copies to stay correct
- **Freezing** finds the layers that have stopped changing
- **Our merge:** stable layers skip the copy or freeze entirely, for less memory at (we hope) the same accuracy
- **Evaluation:** DDG re-checked on a modern baseline; ResNet-50 against PolicyFRZ's numbers; a BERT size ladder for scalability; ablations to attribute each gain

Questions?